

# EDGE DETECTION SYSTEM

## Project Report

**Name: P Roshan**

**Registration Number: 24BAI10682**

Course: CSE3010 – Computer Vision

*Related Experiment: Implement Image Preprocessing and Edge Detection*

# Table of Contents

1. Abstract
2. Objectives
3. Introduction
4. Methodology
5. Implementation
6. Results and Discussion
7. Conclusion
8. References

# 1. Abstract

Edge detection is a fundamental low-level image processing operation that identifies points in a digital image where brightness changes sharply, typically corresponding to object boundaries, surface discontinuities, or texture changes. This project implements and compares four classical edge detection techniques — the Sobel operator, Laplacian of Gaussian (LoG), the Canny edge detector, and Difference of Gaussians (DoG) — using Python and OpenCV. The system takes a grayscale image as input, applies each technique, and produces both individual result images and a consolidated side-by-side comparison. The project demonstrates the trade-offs between simplicity, noise sensitivity, and edge localization accuracy across the four methods.

## 2. Objectives

- To understand and implement fundamental image preprocessing techniques used in computer vision.
- To apply gradient-based and second-derivative-based edge detection operators on a digital image.
- To compare the qualitative performance of Sobel, Laplacian of Gaussian, Canny, and Difference of Gaussians methods.
- To build a reusable, command-line based tool that can process arbitrary input images.

## 3. Introduction

Edges represent one of the most important low-level features in an image because they often correspond to meaningful physical events, such as the boundary of an object, a change in surface orientation, or a variation in illumination. Detecting edges reliably is a prerequisite for many higher-level computer vision tasks such as segmentation, object detection, and feature matching, all of which are covered in later modules of the Computer Vision course.

This project focuses on classical, gradient- and derivative-based edge detection algorithms. These methods operate directly on pixel intensity values and do not require training data, making them fast, interpretable, and a good starting point for understanding how more advanced feature detectors (SIFT, SURF, HOG, etc.) build upon these ideas.

## 4. Methodology

### 4.1 Sobel Operator

The Sobel operator computes an approximation of the image intensity gradient using two  $3 \times 3$  convolution kernels, one for the horizontal direction and one for the vertical direction. The gradient magnitude at each pixel is computed as the Euclidean norm of the horizontal and vertical gradient components, highlighting regions of rapid intensity change.

### 4.2 Laplacian of Gaussian (LoG)

The Laplacian operator computes the second derivative of the image intensity, making it sensitive to regions of rapid intensity change in all directions simultaneously. Because the second derivative amplifies noise, the image is first smoothed with a Gaussian filter before applying the Laplacian, which is why the combined technique is called the Laplacian of Gaussian.

### 4.3 Canny Edge Detector

The Canny algorithm is a multi-stage edge detector consisting of four steps: (1) Gaussian smoothing to reduce noise, (2) computation of intensity gradients, (3) non-maximum suppression to thin out edges to a single-pixel width, and (4) hysteresis thresholding using two thresholds (a lower and an upper bound) to classify and link edge pixels into continuous contours. This produces cleaner and more precisely localized edges than the Sobel or Laplacian methods alone.

## 4.4 Difference of Gaussians (DoG)

The Difference of Gaussians method approximates edge and blob structures by subtracting two versions of the image blurred with Gaussian kernels of different standard deviations. The result approximates the Laplacian of Gaussian at a fraction of the computational cost and is commonly used as a building block in scale-space feature detectors such as SIFT.

## 5. Implementation

The system was implemented in Python using the OpenCV, NumPy, and Matplotlib libraries. The core script, `edge_detection.py`, accepts an image path as a command-line argument (or generates a synthetic demo image if none is provided), applies all four edge detection methods, and saves the results as individual image files along with a combined comparison figure.

### 5.1 Tools and Libraries

| Library      | Purpose                                                       |
|--------------|---------------------------------------------------------------|
| OpenCV (cv2) | Core image processing: filtering, gradients, Canny, Laplacian |
| NumPy        | Array manipulation and numerical operations                   |
| Matplotlib   | Generating the side-by-side comparison figure                 |

### 5.2 Workflow

- Load the input image in grayscale (or generate a synthetic demo image).
- Apply Gaussian blurring to reduce sensitivity to noise.
- Apply each of the four edge detection operators independently.
- Normalize and save each result as an individual PNG file.
- Generate and save a combined comparison figure using Matplotlib.

### 5.3 Command-Line Usage

```
python edge.py im1.png
```

## 6. Results and Discussion

The system was tested on a synthetic image containing geometric shapes (square, circle, triangle, and a diagonal line) with added Gaussian noise, allowing clear visual comparison of each method's noise sensitivity and edge quality.



Figure 1: Comparison of Sobel, Laplacian (LoG), Canny, and Difference of Gaussians on a test image.

### 6.1 Observations

- Sobel produced clear but relatively thick edges and remained somewhat sensitive to the background noise.
- Laplacian of Gaussian detected edges in all directions but was the most sensitive to noise, producing a grainier result.
- Canny produced the cleanest, thinnest, and most continuous edge contours due to non-maximum suppression and hysteresis thresholding.
- Difference of Gaussians gave a smoother, blob-like approximation of edges, useful as a precursor to scale-space feature detectors.

### 6.2 Comparison Table

| Method                  | Noise Sensitivity  | Edge Thickness | Computation Cost |
|-------------------------|--------------------|----------------|------------------|
| Sobel                   | Moderate           | Thick          | Low              |
| Laplacian (LoG)         | High               | Thin, noisy    | Low–Medium       |
| Canny                   | Low (after tuning) | Very thin      | Medium           |
| Difference of Gaussians | Moderate           | Medium         | Low              |

## 7. Conclusion

This project successfully implemented and compared four classical edge detection techniques: Sobel, Laplacian of Gaussian, Canny, and Difference of Gaussians. Among the four, the Canny edge detector produced the most visually clean and precisely localized edges due to its multi-stage design involving non-maximum suppression and hysteresis thresholding, making it the most suitable choice for downstream tasks such as segmentation and object detection. The project reinforced core Computer Vision concepts including convolution-based filtering, gradient computation, and scale-space analysis, and provides a foundation for more advanced feature extraction techniques such as SIFT, SURF, and HOG covered later in the course.